

Model Object Architecture

Mode A: Code-Backed Objects | *Mode B: Schema-Driven Objects*

This document describes the two operational modes of the VistA-Labyrinth model object framework, the architectural rationale behind each, and how they relate to the Registry Viewer reference implementation. It situates the design within contemporary software architecture thinking — primarily Domain-Driven Design (DDD), CQRS, and Event Sourcing — and maps those concepts onto the specific constraints of VistA/MUMPS modernisation.

1. Overview

The VistA-Labyrinth model object framework defines a base class **TVLModelPrototype** that acts as a lifecycle envelope for any domain object in the system. The envelope carries identity, persistence state, lifecycle state, audit timestamps, and an observation seam — independently of whatever domain content the object holds.

Two distinct usage modes emerge from this foundation, driven by whether domain types are implemented as compiled Delphi classes or defined entirely in data:

Type definition	Delphi class with typed properties	TObjectTypeRecord in ModelDesigner DB
Instance	Subclass of TVLModelPrototype with typed fields	TModelObjectInstance with string property bag
Property access	Compiled getter/setter	GetPropertyValue / SetPropertyValue (string)
Validation	DoValidate in subclass + optional external	Required-field check + external (primary validator)
Type registry	Full: class factory, RTTI resolution	Minimal: type name + property descriptor list only
Schema evolution	Requires recompilation	Add/remove in ModelDesigner — no code change
Type safety	Compile-time for domain fields	Runtime only; all values are strings
DDD alignment	Aggregate Root / Rich Domain Model	Anemic Entity + external Domain Services
Best suited for	Stable, complex types with rich behaviour	Rapid prototyping, configurable models, external schemas

2. Contemporary Architectural Context

The Fowler *PEAA* patterns cited historically (Identity Map, Unit of Work, Data Mapper) remain valid descriptions of the mechanics involved. However, the dominant framing in current practice has shifted. Three movements are most relevant to this architecture:

2.1 Domain-Driven Design (DDD) — tactical patterns

Eric Evans's original vocabulary (2003) has been substantially extended by Vaughn Vernon's *Implementing Domain-Driven Design* (2013) and the broader DDD community. The current consensus organises domain objects into:

- **Entities:** objects defined by identity rather than attributes. They have a lifecycle and change over time. TVLModelPrototype is an entity envelope — the GUID provides stable identity independent of database PK assignment.
- **Value Objects:** immutable, defined entirely by attributes, no identity. In VistA-Labyrinth: TInstancePropertyDef is a value object; individual property values in the string bag behave as value objects.
- **Aggregates:** clusters of entities and value objects treated as a consistency unit. The Aggregate Root is the sole entry point for modification. TModelObjectInstance is a single-entity aggregate; its root enforces the MarkDirty / BeginLoad gate invariants.
- **Domain Events:** records of something that happened in the domain. TStateTransitionEntry in VL.Registry.Debug is a lightweight domain event record. A full event sourcing implementation (Phase 2+) would promote these to first-class persisted events.
- **Repositories:** abstractions over persistence that hide storage details from domain logic. TInstanceStore plays this role for Mode B; TModelRegistry acts as an in-memory repository with identity-map semantics.

2.2 CQRS — Command Query Responsibility Segregation

CQRS separates write operations (Commands that mutate state) from read operations (Queries that return data without side effects). In the VistA-Labyrinth context this maps naturally:

Side	Role in VistA-Labyrinth
Command side	ActInstSave, ActInstDelete, ActInstValidate — operations that transition object state and write to the instance DB. Commands go through TModelObjectInstance (the aggregate root) ensuring invariants are enforced before persistence.
Query side	RefreshInstanceList, ShowEnvelope, ShowDomainGrid — reads from the registry and DB that never mutate state. The Registry Viewer is predominantly a query surface.
Read model	TStateTransitionLog maintains a denormalised, ordered view of state history optimised for display — a simple CQRS read model separate from the write model (ModelInstance table).

Note: Full CQRS with separate read/write data stores is not warranted at current scale. The pattern is applied at the code level — separation of mutation and query paths — without physical separation of storage. This is the recommended starting point (Vernon, 2013; Richardson, 2018).

2.3 Event Sourcing

Event Sourcing stores state as an ordered sequence of immutable events rather than as a current-state snapshot. The system state at any point is reconstructed by replaying events from the beginning.

VistA-Labyrinth's current `TStateTransitionLog` is a session-scoped precursor to event sourcing — it records state transitions as ordered entries but does not persist them or use them as the source of truth. The proposed Phase 2 `LifecycleHistory` table moves toward a persistent event log. Full event sourcing — where `ModelInstance` rows are replaced by an append-only event stream — is a Phase 3+ consideration, particularly relevant for VistA audit requirements where complete provenance of every state change may be mandated.

Note: Healthcare regulatory contexts (HIPAA, VistA audit trails) are among the strongest justifications for event sourcing in production. Event sourcing makes the audit log a first-class architectural artifact rather than a secondary concern (Koya, 2025).

2.4 Hexagonal Architecture / Ports and Adapters

Alistair Cockburn's Hexagonal Architecture (also expressed as Clean Architecture by Robert Martin) places domain logic at the centre, with all external concerns — persistence, UI, network — behind adapter interfaces. Dependencies point inward: infrastructure depends on domain, never the reverse.

`TVLModelPrototype` and `TModelObjectInstance` contain no persistence references, no UI references, and no network references. `TInstanceStore`, `TObjectModelData`, and the Registry Viewer form are all adapters around the domain core. This is Hexagonal Architecture applied at the unit level. The `DoLoad/DoSave` stubs in the model classes are the port definitions; `TInstanceStore` is the adapter implementation.

2.5 Modular Monolith over Microservices

Current practice has moved away from reflexive microservice decomposition toward **Modular Monoliths** — single deployable units with clear internal module boundaries that mirror Bounded Contexts (Vernon, 2013; Koya, 2025). This is precisely the VistA-Labyrinth approach: `ModelDesigner`, the Registry, Persistence, and the RPC Gateway are distinct bounded contexts within a coherent desktop application, communicating through well-defined interfaces rather than network calls. Microservice extraction is deferred until a boundary is proven stable enough to warrant the operational overhead.

3. TVLModelPrototype — The Envelope

The envelope metaphor captures the role of the base class precisely: it carries the object, routes it through the system, and tracks its state — but never opens it. In DDD terms it provides the Aggregate Root scaffolding: identity, lifecycle, invariant enforcement (`BeginLoad/EndLoad` gate), and the consistency boundary (`OnStateChanged` seam).

3.1 Responsibilities

- **Identity:** `Int64` DB primary key (0 = not yet persisted) and `TGUID` — the stable Aggregate identity that survives PK reassignment.
- **Persistence state:** `New / Clean / Dirty / Deleted / Conflict`. Tracks synchronisation between in-memory object and DB row.
- **Lifecycle state:** `Draft / Validating / Valid / Invalid / Active / Archived / Error`. Coarse HFSM top-level node. Mode B extends this with data-driven states in Phase 3.
- **Audit:** `CreatedAt`, `ModifiedAt` (UTC), `RowVersion` for optimistic concurrency — standard aggregate versioning (Vernon, 2013).
- **Observation seam:** `OnStateChanged` — single callback subscribed by the registry. No event bus reference in the base class.

3.2 Deliberate exclusions

- No registry reference — objects do not self-register (avoids partial-construction registration hazard).
- No notifier or event bus — registry is the single amplifier.
- No domain fields — content is the subclass's concern entirely.
- No persistence mechanism — DoLoad/DoSave are thin stubs; the Repository (TInstanceStore) owns the dataset lifecycle.

Note: BeginLoad/EndLoad are public so the Repository can suppress dirty-marking during hydration without a friend-pattern workaround. This is the standard solution in Delphi where protected access is by class hierarchy, not by assembly boundary.

4. Mode A — Code-Backed Model Objects

In Mode A each domain type is a Delphi class inheriting from TVLModelPrototype. Typed properties carry domain fields. The class is a Rich Domain Model — it encapsulates both data and behaviour, following the DDD Entity pattern.

4.1 Subclass structure

```
TPatient = class(TVLModelPrototype)
private
    FPatientID    : string;
    FLastName     : string;
    FDateOfBirth  : TDate;
    procedure SetLastName(const AValue: string);
protected
    function DoValidate: Boolean; override;    // domain invariants
    procedure DoLoad; override;                // hydrate from Repository
    procedure DoSave; override;                // write to Repository
public
    function GetTypeName: string; override;    // returns 'TPatient'
    property PatientID    : string read FPatientID    write FPatientID;
    property LastName     : string read FLastName     write SetLastName;
    property DateOfBirth  : TDate read FDateOfBirth write FDateOfBirth;
end;
```

4.2 Registry as Identity Map and Unit of Work

The registry in Mode A provides two classic patterns working together:

- **Identity Map** (Fowler, 2002; Vernon, 2013): guarantees at most one in-memory object per DB identity, preventing split-brain writes. TModelRegistry.FByGUID is the primary identity map; FByID is the secondary.
- **Unit of Work** (Fowler, 2002): registry.Dirty tracks modified objects; TInstanceStore.StoreAll flushes in one coordinated pass, enabling atomic commits across multiple objects.

4.3 Validation tiers in Mode A

Tier	Description
Local structural	DoValidate in subclass. Fast, synchronous. Checks typed field constraints and business invariants within the aggregate boundary.

Client orchestration	Application service layer calls Validate, inspects errors, coordinates cross-aggregate rules (e.g. does this TEncounter reference a valid TPatient?).
External authoritative	RPC Gateway or web service validates against VistA rules. Asynchronous. Sets LifecycleState=Validating during call, Valid or Invalid on response with structured error set.

4.4 Applicable contemporary patterns

- **Rich Domain Model / Aggregate Root** (Evans 2003, Vernon 2013): the typed subclass encapsulates both data and behaviour; all mutations go through the aggregate root.
- **Repository pattern** (Evans 2003, Vernon 2013): TModelRegistry + TInstanceStore together form the Repository — the registry provides identity-map semantics; the store provides persistence.
- **Domain Events** (Vernon 2013): OnStateChanged fires a lightweight domain event on every state transition; TStateTransitionEntry captures it. Full domain event publishing (Phase 3) would route these to external subscribers.
- **Hexagonal Architecture** (Cockburn 2005, Martin 2017): the domain class has no outward dependencies; TInstanceStore is the persistence adapter behind the DoLoad/DoSave port.

5. Mode B — Schema-Driven Model Objects

In Mode B no Delphi class is written for domain types. `TModelObjectInstance` with a `TDictionary` property bag is the complete runtime implementation. This is the contemporary **Anemic Domain Model** deliberately applied — the model holds data; domain logic lives in external services (validation, workflow, RPC). Where Mode A follows Rich Domain Model, Mode B follows the **Domain Service** and **Anti-Corruption Layer** patterns.

5.1 Instance as persistence layer reflection

A Mode B instance is a direct reflection of its DB rows. The property bag maps onto `InstanceProperty` rows one-for-one. The envelope maps onto `ModelInstance` columns. This is the **Row Data Gateway** pattern (Fowler, 2002) enriched with the lifecycle envelope that `TVLModelPrototype` provides — the instance is not merely a data container; it carries state and signals changes.

Note: In DDD terms `TModelObjectInstance` is an Entity (identity matters — GUID is stable) but with an Anemic Domain Model intentionally: behaviour is externalised to Domain Services (the client shell, validation services, RPC Gateway). This is appropriate where the domain logic itself is defined externally (in VistA/MUMPS) and the Delphi layer is an Anti-Corruption translator.

5.2 Property bag design

- Pre-populated at instance creation from `TInstancePropertyDef[]`, injected once from `TObjectModelData`. Follows Prototype Factory pattern — the type definition is the prototype.
- `SetPropertyValue` calls `MarkDirty`, triggering `OnStateChanged` chain through to the registry and observers.
- `PropertySnapshot` returns values ordered by `SortOrder`, matching the canonical order defined in `ModelDesigner`.
- Values for removed properties are preserved in persistence — schema removal does not destroy existing data (additive evolution).

Note: All values are strings. Numeric/date/boolean interpretation is the consumer's responsibility. This is an explicit tradeoff: uniform persistence and transmission over compile-time type safety. JSON-Schema validation at the Gateway layer provides type enforcement without Delphi code changes.

5.3 Validation in Mode B

Tier	Mode B specifics
Local structural	<code>DoValidate</code> checks <code>TypeID</code> is set and all <code>IsRequired</code> properties have non-empty values. Entry point clearly marked in source for extension. Synchronous, no network.
Client orchestration	Shell calls <code>Validate</code> before submission, inspects <code>ValidationErrors</code> , drives lifecycle transition. May apply type-specific rules loaded from configuration or the lifecycle DB.
External authoritative	RPC Gateway validates property bag against VistA business rules. This is the Anti-Corruption Layer translating between the Delphi property bag model and the VistA/MUMPS domain model. Response drives lifecycle transition. Asynchronous.

5.4 Lifecycle states as external data

In Mode B lifecycle states are themselves schema-driven. The `TModelPersistenceState` and `TModelLifecycleState` enums serve as bootstrap states. Production states are loaded from a lifecycle DB:

```
-- Proposed lifecycle schema (Phase 3)
LifecycleDefinition (type_id, state_name, is_terminal, sort_order)
LifecycleTransition (from_state, to_state, trigger, guard_expression)
InstanceLifecycle   (instance_guid, current_state, entered_at)
LifecycleHistory     (instance_guid, from_state, to_state, ts, actor, reason)
```

`LifecycleHistory` is a lightweight event sourcing table — an ordered, append-only record of what happened to each instance. It satisfies healthcare audit requirements without full event sourcing complexity. Promoting to a full event store (Phase 4+) means replacing `ModelInstance` current-state columns with state reconstructed by replaying `LifecycleHistory` entries.

5.5 The client shell as orchestrating application service

In DDD terms the Registry Viewer client is an **Application Service** — it orchestrates domain objects and infrastructure services to fulfil use cases, but contains no domain logic itself. Domain logic lives in three external locations:

Location	Domain logic held
<code>TObjectTypeRecord</code> (schema DB)	What properties exist, which are required, data types, sort order.
Lifecycle DB (Phase 3)	What states exist, what transitions are legal, who can trigger them.
RPC Gateway / VistA	What constitutes a valid domain object under VistA business rules.

The client wires these together and manages the user interaction surface. This is the **Strangler Fig** pattern (Fowler, 2004) applied at the application level: the Delphi shell incrementally replaces legacy VistA workflows by routing through the RPC Gateway while the underlying MUMPS system remains authoritative.

5.6 Applicable contemporary patterns

- **Anemic Domain Model + Domain Services** (Evans 2003; acknowledged by Vernon 2013 as appropriate where behaviour is externally defined): `TModelObjectInstance` holds data; all domain operations are services.
- **Anti-Corruption Layer** (Evans 2003, Vernon 2013): `TInstanceStore` and the RPC Gateway adapter translate between the Delphi property bag model and the VistA/MUMPS domain model, protecting the Delphi layer from upstream model changes.
- **Event Sourcing (precursor)** (Richardson 2018, Koya 2025): `TStateTransitionLog` and the proposed `LifecycleHistory` table form the foundation of an event log. Full event sourcing (append-only, replay-based state reconstruction) is Phase 4.
- **CQRS at code level** (Vernon 2013, Richardson 2018): command path (`ActInstSave` → `TInstanceStore.Store`) is cleanly separated from query path (`RefreshInstanceList` → `TFDQuery`).
- **Prototype Factory**: the type definition (`TObjectTypeRecord` + `TObjectPropertyRecord[]`) is the prototype; `CreateInstance` injects it into a fresh `TModelObjectInstance`.

6. Side-by-Side Comparison

DDD alignment	Rich Domain Model, Aggregate Root	Anemic Entity + Domain Services + Anti-Corruption Layer
Type safety	Compile-time for all domain fields	Runtime only; string bag; consumer interprets values
Schema evolution	Requires Delphi recompilation	Add/edit in ModelDesigner — no code change
Validation	Typed DoValidate + optional external	Required-field check + external (primary authority)
Registry role	Identity Map + lifecycle authority + typed factory	Instance lifecycle manager; no type knowledge required
Persistence	Typed DoSave to schema-specific tables	Uniform: ModelInstance + InstanceProperty for all types
Lifecycle states	Enum in TVLModelPrototype (subclass extendable)	Bootstrap enum; production states from lifecycle DB (Phase 3)
Event sourcing	Domain Events via OnStateChanged; Phase 3: full event store	LifecycleHistory as event log precursor; Phase 4: append-only event store
Client role	Application Service; may contain cross-aggregate rules	Pure orchestrator; all domain logic is external
VistA fit	RPC Command objects, Gateway session lifecycle	FileMan field sets, order entries, configurable forms
Applicable patterns	Aggregate Root, Repository, Domain Events, Hexagonal Architecture, CQRS	Anemic Entity, Domain Services, Anti-Corruption Layer, Event Sourcing (precursor), CQRS, Prototype Factory, Strangler Fig

7. Registry Viewer — Implementation Mapping

Component		Role and pattern
VL.Model.Prototype TVLModelPrototype	/	Aggregate Root scaffolding. Envelope shared by both modes. GUID = stable aggregate identity. OnStateChanged = domain event port.
VL.Registry.Core / TModelRegistry		Identity Map (Mode A) + in-memory Repository (both modes). FByGUID primary; FByID secondary index built post-persist.
VL.Registry.Observer IRegistryObserver	/	Observer port. Non-reference-counted base prevents double-free on shutdown (Delphi-specific lifetime management requirement).
VL.Registry.Debug TStateTransitionLog	/	Lightweight domain event log. Session-scoped precursor to persistent event sourcing. FlushToDatabase stub = Phase 2 hook.
VL.Instance.Model TModelObjectInstance	/	Mode B Entity with Anemic Domain Model. TypeID = aggregate type discriminator. Property bag = Row Data Gateway content layer.

VL.Instance.Persistence / TInstanceStore	Repository adapter for Mode B. Owns SQLite dataset lifecycle. Command-side only (CQRS separation). Phase 2: optimistic concurrency.
DB.Connection / TObjectModelData	Read model for type definitions (CQRS query side). Schema authority owned by ModelDesigner bounded context.
VL.Debug.MainForm	Application Service (DDD). Orchestrates all components. TActionList = command dispatcher; OnUpdate = query-driven UI state.
VL.Debug.TypeSelectForm	Factory dialog. Implements Prototype Factory selection — user picks the prototype; CreateInstance applies it.
VL.Instance.Persistence / TInstanceDatabase	Separate SQLite file = separate bounded context storage. WAL mode + foreign keys = lightweight ACID guarantees.

8. Implementation Plan

Each phase is mapped to the contemporary patterns it realises:

Phase 1 — Current (complete)

Aggregate Root envelope (TVLModelPrototype). Identity Map registry with GUID + ID indexes. Mode B: TModelObjectInstance + TInstanceStore (Repository adapter). Registry Viewer: Application Service shell with TActionList command dispatcher. TStateTransitionLog: domain event log precursor.

Patterns: Aggregate Root, Identity Map, Repository, Row Data Gateway, Domain Events (precursor), CQRS at code level

Phase 2 — Persistence hardening

Optimistic concurrency: RowVersion comparison before UPDATE; PersistenceState=Conflict on mismatch (standard aggregate versioning). TStateTransitionLog.FlushToDatabase: domain event log persistence. TModelRegistry.CreateInstance(TypeID, Defs) factory method. Instance DB schema migration framework (V2+).

Patterns: Unit of Work, Optimistic Concurrency, Event Log

Phase 3 — Lifecycle database

LifecycleDefinition + LifecycleTransition tables: type-specific state machines in data. TLifecycleStore: state machine loading, transition validation, LifecycleHistory persistence (append-only event log). Viewer: lifecycle editor; transition history per instance.

Patterns: Event Sourcing (lightweight), HFSM in data, Audit Trail

Phase 4 — External validation and RPC integration

RPC Gateway adapter: submit property bag, receive structured errors, update LifecycleState. Async validation: Validating state during call. Anti-Corruption Layer between Delphi property bag and VistA domain model. Mode A: TRPCCCommand as first typed TVLModelPrototype subclass.

Patterns: Anti-Corruption Layer, Domain Events (outbound), Strangler Fig, Rich Domain Model (Mode A)

Phase 5 — Full event sourcing (optional)

Replace `ModelInstance` current-state columns with state reconstructed by replaying `LifecycleHistory`. Append-only event store; time-travel queries; complete audit provenance for HIPAA/VistA compliance.

Patterns: Event Sourcing, CQRS (physical read/write separation), Projection-based Read Models

9. Key Design Decisions and Rationale

BeginLoad/EndLoad are public

The Repository (`TInstanceStore`) is not a `TVLModelPrototype` subclass, so protected access would require a friend-pattern workaround. Public visibility with clear documentation is preferable. This is the standard Delphi resolution to the "package-private" access problem that Java and C# solve with assembly-level visibility.

OnStateChanged is a single callback, not multicast

The registry is the sole subscriber at the base level, then routes to `IRegistryObserver` implementations. This prevents cascading notification overhead when many objects change state together, and keeps the aggregate root ignorant of observer count.

TBaseRegistryObserver inherits from TObject, not TInterfacedObject

Mixing Free and interface reference counting on the same object causes `EInvalidPointer` on shutdown. Non-reference-counted interface implementation (`_AddRef/_Release` return -1) gives explicit lifetime control to the owning form. This is the standard Delphi pattern for observer/listener objects that implement interfaces without surrendering lifetime ownership.

Property bag values are all strings

Uniformity simplifies persistence (one `InstanceProperty` table for all types), UI rendering (`TStringGrid` with no type-specific editors), and external transmission (JSON-serialisable without schema knowledge). Type enforcement is deferred to the external validation layer (JSON Schema at the RPC Gateway). This is an intentional Anemic Domain Model tradeoff, appropriate where the authoritative type system is in VistA/MUMPS, not in Delphi.

Separate instance DB from schema DB

`ModelDesigner` owns the schema DB. Mixing instance data into it couples two independently deployable bounded contexts. Separation follows the DDD principle that bounded contexts do not share data models.

TActionList for all UI operations

Actions centralise caption, enabled state, and shortcut in one place. `OnUpdate` handlers driven by Delphi's idle cycle implement the query side of CQRS at the UI level: button state is derived from application state, never set imperatively.

Lifecycle states separate from instance data

Workflow position and property values evolve independently. This follows the Event Sourcing principle that state is a derived view over an event sequence, not a primary record. Separation enables: workflow definition changes without touching instance records; lifecycle history as a first-class audit log; different types with entirely different state machines.

Modular Monolith over microservices

ModelDesigner, Registry, Persistence, and RPC Gateway are separate bounded contexts within one deployable unit. Current VistA modernisation scale does not justify the operational complexity of distributed services. Boundaries are preserved in code (separate units, no cross-boundary direct field access) so extraction to services is straightforward if scale demands it.

10. References

References are organised by recency. Foundational works whose patterns remain in active use are included alongside contemporary sources that reflect current practice.

Foundational (patterns remain current)

- Evans, Eric.** *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Addison-Wesley, 2003. Defines Entities, Value Objects, Aggregates, Repositories, Domain Events, Bounded Contexts, Anti-Corruption Layer, and Ubiquitous Language. The vocabulary used throughout this document derives from Evans.
- Fowler, Martin.** *Patterns of Enterprise Application Architecture*. Addison-Wesley, 2002. Identity Map, Unit of Work, Data Mapper, Row Data Gateway. These mechanics underlie the registry and persistence layer. Fowler's bliki (martinfowler.com) continues to be updated with contemporary commentary on these patterns.
- Cockburn, Alistair.** "Hexagonal Architecture." alistair.cockburn.us, 2005. Ports and Adapters pattern: domain at the centre, infrastructure adapters at the boundary. Applied at the unit level throughout VistA-Labyrinth.
- Fowler, Martin.** "StranglerFigApplication." martinfowler.com, 2004. Incremental replacement of legacy system functionality by routing through a new facade. Core pattern for VistA/MUMPS modernisation via the RPC Gateway.

Contemporary (2013 — present)

- Vernon, Vaughn.** *Implementing Domain-Driven Design*. Addison-Wesley, 2013. Practical implementation of Evans's patterns including Aggregate design, optimistic concurrency with RowVersion, Domain Events, CQRS implementation guidance, and the distinction between Rich Domain Models and Anemic Domain Models applied deliberately. The primary contemporary DDD reference.
- Martin, Robert C.** *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Prentice Hall, 2017. Dependency Rule: source code dependencies point inward toward domain. Reinforces Hexagonal Architecture at larger scale. Applied in VistA-Labyrinth: infrastructure units (TInstanceStore, TObjectModelData) depend on domain units (VL.Model.Prototype), never the reverse.
- Richardson, Chris.** *Microservices Patterns*. Manning, 2018. CQRS and Event Sourcing implementation patterns, Saga pattern for distributed consistency, and the Strangler Fig at service level. Richardson's microservices.io remains an active reference site (2026). The CQRS separation applied at code level in VL.Debug.MainForm follows Richardson's guidance on starting with logical rather than physical CQRS separation.
- Wlaschin, Scott.** *Domain Modeling Made Functional*. Pragmatic Bookshelf, 2018. Demonstrates that DDD tactical patterns (Entities, Value Objects, Aggregates) apply outside object-oriented languages and that type systems can enforce domain invariants at compile time. Relevant to VistA-Labyrinth Mode A typed properties and the Mode B deliberate tradeoff of type safety for schema flexibility.
- Koya, Bhargava.** "Mastering CQRS and Event Sourcing in .NET 8." Medium, 2025. Practical implementation of Event Sourcing for audit trail requirements in regulated industries. Directly relevant to VistA healthcare context. The LifecycleHistory table proposed in Phase 3 follows the lightweight event log pattern described here.
- Kundalia, Raj.** "Demystifying Domain-Driven Design: Principles, Practice and Relevance in Modern Software Architecture." Medium, 2025. Confirms DDD relevance in current practice and the move toward Modular Monolith as the preferred starting point over microservices.
- Growin Engineering.** "Event Driven Architecture Done Right: How to Scale Systems with Quality in 2025." growin.com, 2025. Survey of Event Sourcing, CQRS, and Domain Events in production systems at Netflix, Uber, LinkedIn. Confirms these as mainstream patterns, not experimental, in contemporary architecture.

OSEHRA / FOIA VistA. VistA Electronic Health Record System documentation and source code. Open Source Electronic Health Record Alliance, ongoing. FileMan data dictionary structures and RPC definitions inform the property bag design and the Anti-Corruption Layer requirements between the Delphi model and the VistA/MUMPS domain.

Embarcadero Technologies. Delphi 12 / RAD Studio documentation. TActionList architecture, FireDAC SQLite driver, non-reference-counted interface implementation pattern (TInterfacedObject vs TObject + manual IUnknown). docs.embarcadero.com, 2024.

VistA-Labyrinth Project — internal architecture document. Subject to revision as implementation progresses.